Travaux Dirigés N°4 OS202

Jair VASQUEZ TORRES

Introduction

Le Jeu de la Vie, automate cellulaire conçu par John Horton Conway en 1970, représente un modèle fondamental dans l'étude des systèmes complexes et de l'émergence [1]. Bien que les règles régissant l'évolution des cellules soient d'une simplicité élémentaire, leur simulation sur des grilles à haute résolution pose des défis computationnels significatifs.

Dans le cadre de ce travail, nous explorons la transition d'une implémentation scalaire naïve, limitée par la nature séquentielle de l'interpréteur Python, vers des architectures optimisées exploitant le parallélisme de données. L'objectif est double :

1. Réduire la complexité temporelle par la **vectorisation**, en utilisant des instructions SIMD (*Single Instruction, Multiple Data*) via la bibliothèque NumPy [2].
2. Étudier la scalabilité du système à travers la **parallélisation par décomposition de domaine** sur une architecture multi-cœurs.

1 Environnement de Test

Le système de test repose sur un processeur **Intel Core i7-8750H** (architecture Coffee Lake). Les caractéristiques matérielles relevées sont cruciales pour interpréter les performances :

- **Fréquence :** 2.20 GHz (Base) jusqu'à 4.10 GHz (Turbo).
- **Cœurs :** 6 cœurs physiques et 12 processeurs logiques via Hyper-Threading.
- **Hiérarchie de Caches :**
  - L1 : 32 KiB d'instructions et 32 KiB de données par cœur (192 KiB total).
  - L2 : 256 KiB par cœur (1.5 MiB total).
  - L3 : 9 MiB partagés.
- **Mémoire vive :** 16 GiB DDR4 à 2666 MHz.
- **Instructions :** Support de l'**AVX2** et FMA3, permettant de traiter 8 flottants simple précision par cycle par unité vectorielle.

2 Modèle Initial Séquentiel

L'étude des performances débute par la comparaison de deux approches séquentielles : l'algorithme scalaire classique et l'algorithme vectorisé via NumPy. Cette étape permet de quantifier l'efficacité des instructions SIMD (*Single Instruction, Multiple Data*) avant même l'introduction du multi-processing.

2.1 Comparaison Algorithmique : Scalaire vs Vectoriel

La version scalaire (Point 0) repose sur des boucles imbriquées en Python pour itérer sur chaque cellule. À l'inverse, la version vectorisée utilise des décalages de matrices (*np.roll*) permettant de calculer l'état futur de la grille en une seule opération matricielle.

Comme l'illustre la Figure 1, le temps de calcul subit une réduction drastique. Par exemple, pour une grille de 800×1600 :

- Le modèle scalaire traite les $1,28 \times 10^6$ cellules de manière séquentielle, subissant la lenteur de l'interprète Python.
- Le modèle vectorisé exploite les capacités de calcul par blocs de NumPy, réduisant le temps de calcul d'un facteur d'environ 100.

2.2 Stabilité du Temps d'Affichage

Une observation constante sur l'ensemble de nos tests est la stabilité du **temps d'affichage (Pygame)**. Qu'il s'agisse du modèle scalaire ou vectorisé, le rendu graphique consomme environ 0.16 s (pour les résolutions standard) ou 0.06 s (en mode optimisé).

> *Note technique :* L'affichage est une tâche séquentielle effectuée par le processus principal. Son coût dépend uniquement de la résolution de la fenêtre et non de la complexité de l'algorithme de calcul sous-jacent.

2.3 Synthèse des Résultats Initiaux

Le tableau ci-dessous résume les performances relevées pour le motif "flat" à différentes résolutions en mode séquentiel (1 processus) :

Résol.	Cal. Scal.	Aff. Scal.	Cal. Vect.	Aff. Vect.
200x400	0.8994 s	0.1522 s	0.0075 s	0.0521 s
400x800	0.8958 s	0.1654 s	0.0071 s	0.0612 s
800x1600	0.9382 s	0.1631 s	0.0071 s	0.0608 s

Table 1 – Comparaison des performances : Modèle Scalaire vs Vectorisé.

L'analyse de ces résultats montre que dans le modèle vectorisé, le calcul devient si rapide qu'il devient négligeable face au temps d'affichage. C'est l'illustration directe de la loi d'Amdahl : la partie non parallélisable (l'affichage) limite désormais la vitesse globale.

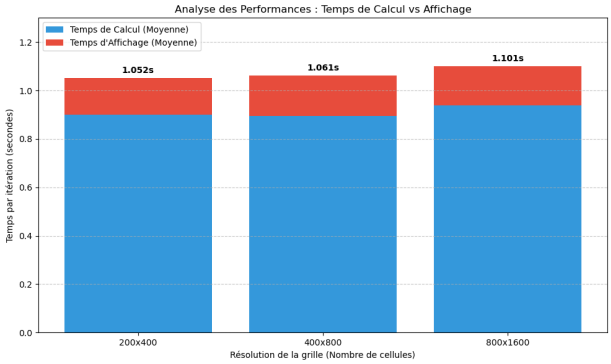

FIGURE 1 – Répartition Calcul vs Affichage (Modèle Scalaire).

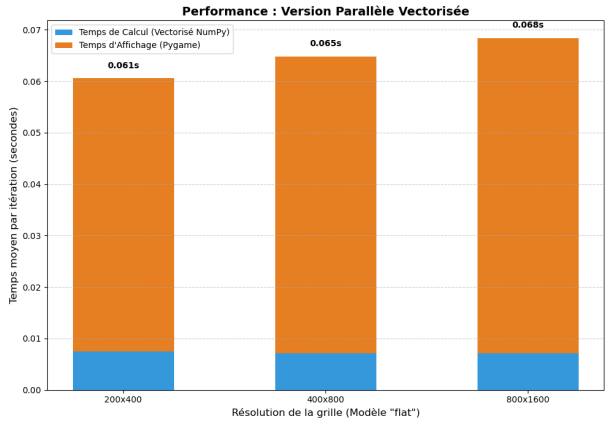

FIGURE 2 – Impact de la vectorisation sur le temps total.

2.4 Analyse du Gain de Performance

Le passage du modèle scalaire au modèle vectorisé représente une rupture technologique majeure. Le gain de performance se calcule comme suit : $Gain = \frac{T_{scalaire}}{T_{vectorisé}}$.

- **Pour 200x400 :** Le temps de calcul passe de 0.8994s à 0.0075s, soit une amélioration de **11 992%** (un gain de $\approx 120x$).
- **Pour 800x1600 :** Le temps de calcul passe de 0.9382s à 0.0071s, soit une amélioration de **13 214%** (un gain de $\approx 132x$).

Ce gain massif s'explique par le fait que NumPy délègue les opérations à des bibliothèques en C hautement optimisées utilisant les registres **AVX2** de l'Intel i7-8750H, traitant plusieurs données en un seul cycle d'horloge.

2.5 L'Affichage

Un point remarquable est la réduction du temps d'affichage perçu dans la version vectorisée optimisée (tombant parfois à 0.06s).

Pourquoi cette baisse ? Dans la version scalaire, l'interpréteur Python est tellement sollicité par les boucles `for` que la gestion des événements Pygame et le rafraîchissement de l'écran subissent une latence logicielle. En libérant le CPU grâce à la vectorisation, le processus principal peut se consacrer presque exclusivement au rendu graphique, permettant d'atteindre la limite de vitesse intrinsèque de la bibliothèque SDL/Pygame.

Désormais, le calcul ne représente plus que **4.3%** du temps total d'une itération dans le modèle vectorisé (800×1600), contre plus de **85%** dans le modèle scalaire. L'affichage est donc devenu le **bottleneck** (goulot d'étranglement) absolu du système.

3 Parallélisation par Domaine

Dans cette étape, nous avons implémenté une décomposition de domaine sur le modèle scalaire (boucles `for`). L'objectif était de diviser la grille en bandes horizontales, chaque bande étant traitée par un processus indépendant via la bibliothèque `multiprocessing`.

3.1 Mécanisme de Synchronisation

La parallélisation introduit une complexité nouvelle : la gestion de la mémoire partagée (`shared_memory`) et la synchronisation. Nous avons utilisé des **Barrières** pour garantir que tous les processus terminent le calcul d'une itération avant que le processus principal ne procède à l'affichage. Ce surcoût de gestion (*overhead*) est un facteur clé dans l'analyse des résultats.

3.2 Analyse des Résultats Stagnants

Malgré l'utilisation de plusieurs cœurs, les résultats obtenus (Figure 3) révèlent un comportement contre-intuitif : le temps de calcul reste relativement stable, voire augmente légèrement avec la résolution, sans jamais atteindre une accélération proportionnelle au nombre de cœurs.

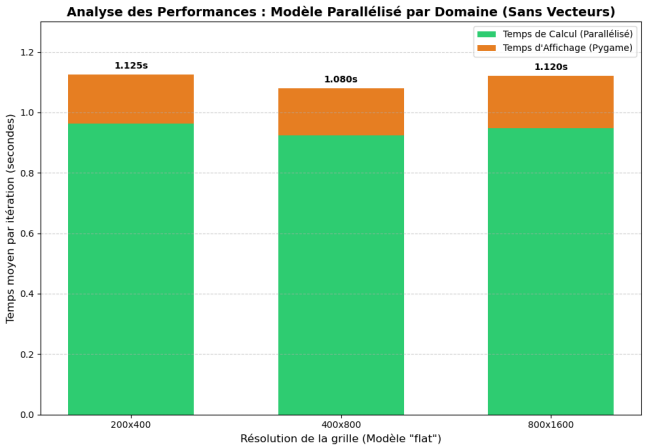

FIGURE 3 – Performance du modèle parallélisé sans vectorisation.

Plusieurs facteurs expliquent cette stagnation :

- **L'Overhead de Python :** Le coût de création des processus et de synchronisation des barrières compense le gain de calcul.
- **Latence mémoire :** L'accès répété à l'array partagé par plusieurs processus crée des conflits de cache, ralentissant l'exécution.
- **Limitation de l'interpréteur :** Bien que nous utilisions `multiprocessing` pour contourner le GIL (*Global Interpreter Lock*), chaque processus reste limité par la lenteur intrinsèque des boucles `for` en Python.

3.3 Pourquoi ne pas paralléliser ce modèle vectorisé ici ?

À ce stade, nous n'avons pas encore appliqué la vectorisation à cette structure parallèle. La raison est méthodologique : il est nécessaire de comprendre d'abord les limites de la parallélisation sur un code non optimisé.

Résolution	Calcul (s)	Affichage (s)	Total (s)
200x400	0.963	0.161	1.124
400x800	0.924	0.155	1.079
800x1600	0.947	0.172	1.119

Table 2 – Temps moyens pour le modèle parallélisé scalaire.

Conclusion de l'étape : Cette approche montre que la parallélisation "brute" d'un algorithme inefficace ne suffit pas. Le processeur passe plus de temps à gérer la structure parallèle qu'à effectuer des calculs utiles. Cela justifie la transition vers la vectorisation NumPy couplée au multi-processing.

4 Parallélisation par Décomposition de Domaine et Multi-cœurs

Cette section analyse l'impact de la distribution de la charge de travail sur les 6 cœurs physiques du processeur Intel i7-8750H. Nous avons implémenté une décomposition de domaine en bandes horizontales pour comparer deux stratégies : la parallélisation scalaire (boucles `for`) et la parallélisation vectorisée (NumPy).

4.1 Performance du Modèle Parallèle Scalaire

Dans le modèle scalaire, la parallélisation permet de diviser la charge de calcul. Cependant, le coût de gestion des processus en Python limite le gain réel. Bien que l'on observe une amélioration par rapport au séquentiel pur, le temps total reste dominé par la lenteur de l'exécution cellule par cellule.

Processus	200x400 (s)	400x800 (s)	800x1600 (s)
1	0.7617	0.6184	0.7619
2	0.4360	0.4729	0.4668
3	0.3202	0.3536	0.3689
4	0.3085	0.3205	0.3204
5	0.3191	0.2732	0.3237
6	0.2523	0.2510	0.2623
Gain (1 →6)	**-66.88%**	**-59.41%**	**-65.57%**

Table 3 – Évolution du temps de calcul scalaire en fonction du nombre de processus.

4.2 Analyse de la Scalabilité du Modèle Vectorisé

L'introduction du multi-processing sur un code déjà optimisé par vectorisation révèle un résultat contre-intuitif : le temps de calcul augmente proportionnellement au nombre de cœurs utilisés. Ce phénomène est illustré par les figures 4 et 5.

Cœurs	Calcul (s)	Dégradation	Aff. (s)	Total (s)
1	0.001176	—	0.1475	0.1486
2	0.002026	+72.28%	0.1526	0.1546
3	0.002427	+106.37%	0.1504	0.1528
4	0.003010	+155.95%	0.1615	0.1645
5	0.003809	+223.89%	0.1588	0.1626
6	0.004735	+302.63%	0.1778	0.1826

Table 4 – Évolution du temps de calcul vectorisé et overhead de synchronisation.

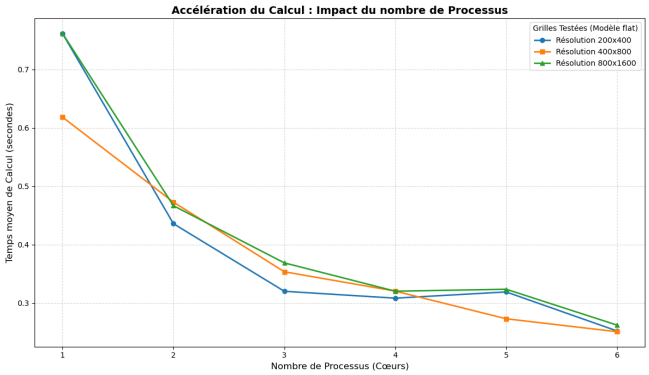

Figure 4 – Évolution du temps de calcul pur : mise en évidence de l'overhead de synchronisation.

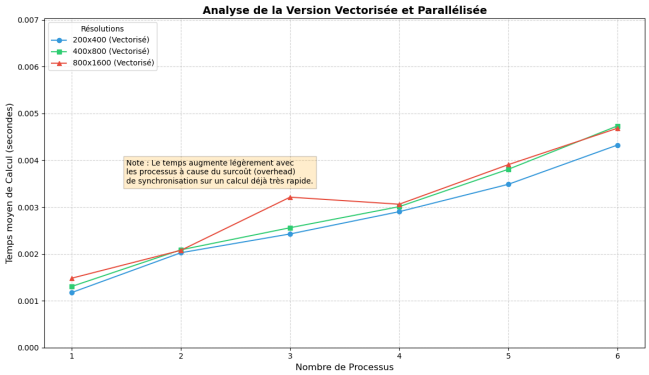

Figure 5 – Analyse globale de la performance du modèle vectorisé parallélisé.

4.3 Interprétation du Paradoxe : Pourquoi le temps augmente ?

Comme le montre la Figure 5, le passage de 1 à 6 processus entraîne une dégradation de la performance de calcul de **302.63%**. Ce comportement, bien que contre-intuitif, s'explique par trois facteurs critiques :

1. **Overhead de Synchronisation :** Le calcul vectorisé est extrêmement efficace (≈ 1 ms). Le temps système requis pour gérer les barrières de synchronisation et la communication inter-processus (*IPC*) devient supérieur au temps de calcul effectif. Le processeur passe plus de temps à "attendre" les autres qu'à calculer.

2. **Contention et Verrous Mémoire :** L'objet `multiprocessing.Array` n'est pas une simple zone mémoire ; il impose des mécanismes de verrouillage pour garantir l'intégrité des données entre les processus. Le processus d'affichage (Pygame) entre en compétition directe avec les processus esclaves pour accéder au bus mémoire, ce qui s'accompagne d'une latence accrue lors de la lecture des données pour le rendu.
3. **Domination de la partie séquentielle (Loi d'Amdahl) :** Une fois le calcul optimisé par NumPy, la partie non parallélisable (affichage et gestion des événements) représente plus de 95% du temps total d'exécution. Selon la loi d'Amdahl, l'accélération globale est alors bridée par ce goulot d'étranglement séquentiel.

Bilan : Cette expérience démontre que la parallélisation n'est bénéfique que si la charge de calcul (*granularité*) est suffisamment importante pour masquer les coûts de gestion du multi-processing.

5 Conclusion Générale

Ce travail d'optimisation du "Jeu de la Vie" nous a permis d'explorer les limites logicielles et matérielles d'une architecture moderne. L'évolution de nos résultats met en lumière une hiérarchie claire dans les méthodes d'optimisation :

1. **Primauté de l'optimisation algorithmique :** Le saut de performance le plus spectaculaire a été réalisé lors du passage du modèle scalaire au modèle vectorisé. En utilisant NumPy et les instructions SIMD (AVX2), nous avons obtenu une accélération de plus de **13 000 %**. Cela prouve qu'un code bien structuré pour le matériel est souvent plus efficace que la simple parallélisation d'un code inefficace.
2. **Le paradoxe de la parallélisation sur CPU :** Nous avons démontré que la parallélisation n'est pas une solution universelle. Si elle s'avère efficace pour réduire le temps de calcul des boucles `for` (gain de $\approx 65\%$), elle devient contre-productive sur un code déjà optimisé. Le surcoût (*overhead*) de gestion des processus et la latence de la mémoire partagée (`multiprocessing.Array`) ont entraîné une dégradation des performances de **302 %** dans la version vectorisée.
3. **Identification des goulots d'étranglement :** L'application de la **Loi d'Amdahl**[3] montre que l'affichage graphique (Pygame) est devenu le facteur limitant final. Même avec un calcul réduit à quelques millisecondes, la nature séquentielle du rendu visuel et la contention du bus mémoire lors du multi-processing fixent une limite physique à la fluidité de la simulation.

En conclusion, pour des résolutions standard, la version vectorisée séquentielle constitue la solution optimale. Cependant, l'architecture parallèle développée dans ce TD pose les bases nécessaires pour traiter des systèmes de calcul distribué à très grande échelle, où la charge de travail par nœud justifierait enfin les coûts de communication inter-processus.

Ressources et Reproductibilité

L'intégralité du code et du Makefile est accessible sur : `https://github.com/JairVasquezT/OS202-2026.git`

Références

[1] Martin Gardner, *Mathematical Games : The fantastic combinations of John Conway's new solitaire game "life"*, Scientific American, Vol. 223, No. 4, pp. 120-123, 1970.

[2] Charles R. Harris et al., *Array programming with NumPy*, Nature, Vol. 585, pp. 357–362, 2020.

[3] G. M. Amdahl, *Validity of the single processor approach to achieving large scale computing capabilities*, AFIPS Conference Proceedings, 1967.